CONTACT 02 / 01

RAG SUPPORT

Two models power the RAG pipeline.

Embedding model

Converts passages and questions into vectors for search.

Generation model

Uses the retrieved passages to write an answer.

Model selection

Compare retrieval and answer quality, latency, usage and cost.

> Codex is our engineering tool. Its model choice is separate from these two runtime roles.

Code `workbench/teaching/provider_connection.py:19`

CONTACT 02 / 02

RAG SUPPORT

What does 1,536 dimensions mean?

A vector is a list

1,536 dimensions means 1,536 numbers for each passage or question.

One compatible space

Use the same embedding model and dimension setting for both.

Storage and quality

More dimensions use more vector storage. Measure the retrieval benefit.

> Dimensions measure vector length. Tokens measure text.

Code `workbench/teaching/contact2/config.py:20`

CONTACT 02 / 03

RAG SUPPORT

Why chunk, and how should we split?

Retrieve the relevant passage, keep useful context and avoid sending the whole document.

Fixed-size

Split by a character or token limit, often with overlap. Simple; can cut across a rule.

Recursive

Try paragraph, line and word boundaries before smaller splits, within a size limit.

Semantic

Use meaning or topic changes to choose boundaries. More processing; quality needs testing.

> Demo: size-based, word-aware chunks · 320 characters · 64-character target overlap.

Code `workbench/teaching/contact2/indexing/chunking.py:50`

CONTACT 02 / 04

RAG SUPPORT

Similarity, top-k and hybrid search.

Cosine + threshold

Higher cosine means more similar. The cutoff rejects weak vector matches.

Top-k + hybrid

Top-k limits results. Hybrid combines vector and keyword rankings.

Access filters

Tenant and role checks restrict which passages can be returned.

> Demo: top-k 3 · cosine cutoff 0.4, uncalibrated. A hybrid score is not cosine.

Code `workbench/teaching/contact2/querying/retrieval.py:14`

CONTACT 02 / 05

RAG SUPPORT

Retrieved does not mean supported.

Relevance

A similar passage may still omit the rule the question needs.

Context budget

Instructions, history, tools, passages and output all need room.

Evidence checks

Check the source, version and passage behind the answer.

> A citation matters only when its text supports the claim.

Code `workbench/teaching/contact2/querying/answer.py:9`

CONTACT 02 / 06

CODEX ENGINEERING

Choose the model and reasoning effort.

Model

Match capability, speed and cost to the engineering task.

Effort

Low / Medium / higher allocate different reasoning budgets, where supported.

Evidence

Judge the patch and checks, not how long the model thought.

> More reasoning does not grant more permission. Newer does not necessarily mean costlier.

CONTACT 02 / 07

CODEX ENGINEERING

A clear task before an implementation.

Prompt

Give the objective, relevant files, constraints and acceptance evidence.

Plan mode

Investigate the code and agree the approach before editing.

Implementation

Make the bounded change, verify it and report actual results.

> “Plan first” is an instruction. Selecting Plan mode is a client control.

CONTACT 02 / 08

CODEX ENGINEERING

Repository instructions and task context.

AGENTS.md

Repository conventions, commands and boundaries, scoped by directory.

Current context

The conversation, supplied files and tool results available to this task.

Handover

Record files, checks, results and the next step for a fresh task.

> Persistent files help only when their relevant instructions or content are loaded.

CONTACT 02 / 09

CODEX ENGINEERING

A skill is a reusable procedure.

Review a change

Inspect a real diff, check risks and report actionable findings.

Check a browser journey

Verify visible behavior with isolated test data and evidence.

Ingest a policy

Call the existing pipeline, verify the result and report what happened.

> AGENTS.md gives repository guidance. A skill describes how to perform a kind of task.

CONTACT 02 / 10

CODEX ENGINEERING

Sandbox, approvals and secrets.

Sandbox

Defines which files and network resources the agent may access.

Approval review

Determines when an action needs review and who reviews it.

Untrusted input

Documents and tickets cannot grant permissions or request secret access.

> Changing the reviewer does not expand the sandbox.

CONTACT 02 / 11

CODEX ENGINEERING

Built-in tools, MCP and application tools.

Built-in tools

Codex reads files, edits code and runs allowed commands.

External MCP

A connected server exposes operations, such as documentation search.

Application tools

Relay calls a shipment function while answering a support question.

> Inspect the available operation and its scope before invoking it.

CONTACT 02 / 12

CODEX ENGINEERING

Loops need feedback and a finish line.

Tool loop

Model requests a tool → tool result returns → model continues.

Engineering loop

Reproduce → patch → test → inspect feedback → improve.

Goal continuation

Pursue a thread objective within lifecycle, permission and budget limits.

> Stop at success, a budget limit, missing approval or an external blocker.

CONTACT 02 / 13

CODEX ENGINEERING

Caching is not conversation memory.

Prompt cache

The provider may reuse eligible repeated input. Check reported cached tokens.

Embedding reuse

Relay reuses vectors for unchanged chunks during a repeat import.

History and state

Earlier turns affect a request only when supplied or explicitly continued.

> An answer cache is another mechanism. Relay does not implement one here.

Code `workbench/teaching/contact2/indexing/pipeline.py:20` `workbench/teaching/memory_reference.py:9`

CONTACT 02 / 14

CODEX ENGINEERING

From an isolated change to a reviewable PR.

Worktree + tests

Keep live edits isolated. Reproduce the issue and check behavior and boundaries.

Browser verification

Exercise the visible journey using disposable data and inspect its result.

Diff + PR

Review scope, checks and remaining limits before proposing the change.

> The sequence is plan → reproduce → test → patch → browser check → diff review → PR.

CONTACT 02 / 15

CODEX ENGINEERING

The same task can run non-interactively.

Bounded input

Supply a concrete objective, repository context and acceptance criteria.

Controlled execution

Keep permissions, budgets and stopping conditions explicit.

Reviewable output

Capture the result, diff and actual verification evidence.

> The prepared CLI run is executed evidence. SDK/cloud examples are reference material.